

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №1

«Методы сортировки»

Вариант 5 / 5 / 5 / 5

Выполнил:
студент 104 группы
Ильин А.Д.

Преподаватель:
Гуляев Д.А.

Москва
2026

Содержание

Постановка задачи	2
Результаты экспериментов	3
Структура программы и спецификация функций	4
Отладка программы, тестирование функций	5
Анализ допущенных ошибок	6
Список цитируемой литературы	7

Постановка задачи

В данной работе требуется реализовать два алгоритма сортировки одномерного массива целых чисел — пузырьковую сортировку и быструю сортировку с разбиением Ломута — и провести экспериментальное сравнение их эффективности. Оба метода должны упорядочивать элементы по возрастанию.

Для исследования необходимо создать несколько вариантов исходных данных:

- полностью упорядоченный массив;
- массив, отсортированный в обратном порядке;
- два различных массива со случайным расположением элементов (заполняются с помощью генератора случайных чисел).

Сравнение алгоритмов выполняется на одних и тех же наборах данных при разных размерах массива: 10, 100, 1000 и 10000 элементов. Для каждого размера и типа исходного массива проводится несколько запусков (например, 5), после чего усредняются две ключевые метрики:

- количество сравнений элементов;
- количество перемещений (обменов или присваиваний) элементов в процессе сортировки.

Полученные результаты позволяют оценить зависимость трудоёмкости алгоритмов от объёма и характера входных данных и сделать выводы об их применимости в различных условиях.

Результаты экспериментов

В ходе эксперимента были получены следующие данные.

Пузырьковая сортировка. Количество сравнений для всех типов массивов при фиксированном n остаётся неизменным и равно $\frac{n(n-1)}{2}$, что полностью соответствует теоретической оценке. Количество перемещений зависит от исходной упорядоченности:

- на упорядоченном массиве перемещения отсутствуют;
- на полностью обратном массиве число перемещений максимально и также равно $\frac{n(n-1)}{2}$;
- на случайных массивах число перемещений принимает промежуточные значения (например, при $n = 10$ в среднем около 20–25).

Быстрая сортировка с разбиением Ломута. Для случайных массивов число сравнений и перемещений растёт пропорционально $O(n \log n)$, что заметно при увеличении n : так, при переходе от $n = 1000$ к $n = 10000$ рост составляет примерно 10–12 раз, а не 100 раз, как в случае квадратичной зависимости. На упорядоченных массивах (возрастающем и убывающем) наблюдается худший случай: количество сравнений приближается к $\frac{n(n-1)}{2}$, что соответствует квадратичной сложности. Это объясняется выбором последнего элемента в качестве опорного.

Полученные экспериментальные данные подтверждают теоретические оценки рассматриваемых алгоритмов сортировки.

n	Параметр	Номер сгенерированного массива				Среднее значение
		1	2	3	4	
10	Сравнения	45	45	27	23	35
	Перемещения	0	5	11	6	5.5
100	Сравнения	4950	4950	606	672	2794.5
	Перемещения	0	50	324	272	161.5
1000	Сравнения	499500	499500	10550	11208	255189.5
	Перемещения	0	500	5038	5588	2781.5
10000	Сравнения	49995000	49995000	174260	179592	25085963
	Перемещения	0	5000	67004	71881	35971.25

Таблица 1: Результаты работы сортировки Quick sort (Lomuto)

Структура программы и спецификация функций

- `genOrdered(int a[], int n)` – заполняет массив `a` упорядоченными по возрастанию числами от 1 до `n`.
- `genReversed(int a[], int n)` – заполняет массив `a` числами в обратном порядке: `n, n-1, ..., 1`.
- `genRand1(int a[], int n)` – заполняет массив `a` случайными числами от 0 до 999 (первый вариант случайного массива).
- `genRand2(int a[], int n)` – заполняет массив `a` случайными числами от 0 до 999 (второй вариант случайного массива).
- `bubbleSort(int a[], int n)` – реализует пузырьковую сортировку массива `a` по возрастанию; в процессе работы увеличивает глобальные счётчики `'cmp'` (число сравнений) и `'mov'` (число обменов).
- `partition(int a[], int lo, int hi)` – статическая вспомогательная функция для быстрой сортировки; выполняет разбиение подмассива `a[lo..hi]` по схеме Ломута (опорный элемент – последний). Возвращает индекс, на который опорный элемент становится после разбиения. Увеличивает `'cmp'` при каждом сравнении и `'mov'` при каждом обмене.
- `qs(int a[], int lo, int hi)` – статическая рекурсивная функция быстрой сортировки; сортирует подмассив `a[lo..hi]` с помощью вызова `'partition'` и рекурсивных вызовов для левой и правой частей.
- `quickSort(int a[], int n)` – обёртка для вызова быстрой сортировки всего массива; вызывает `'qs'` с границами 0 и `n-1`.
- `copyArray(int dest[], int src[], int n)` – копирует содержимое массива `'src'` в массив `'dest'`; используется для создания копий исходных массивов перед каждым запуском сортировки.

Отладка программы, тестирование функций

Отладка и тестирование алгоритмов проводились с помощью автоматической проверки упорядоченности массива после сортировки. Для этого использовалась вспомогательная функция:

```
int is_sorted(const int *arr, int n) {
    for (int i = 1; i < n; ++i) {
        if (arr[i] < arr[i - 1]) {
            return 0;
        }
    }
    return 1;
}
```

Рассматривались следующие случаи:

- * Граничные случаи: пустой массив ($n = 0$) и массив из одного элемента. Оба алгоритма корректно обработали эти ситуации (сортировка не вносила изменений, функция `'issorted'`1).
- * Уже отсортированный массив по возрастанию: например, 1, 2, 3, 4, 5. После сортировки массив остался неизменным, `'issorted = 1'`.
- * Массив, отсортированный по убыванию (обратный порядку): 5, 4, 3, 2, 1. Оба метода успешно переупорядочили его в 1, 2, 3, 4, 5.
- * Случайная последовательность: например, 3, 1, 4, 1, 5, 9, 2, 6, 5. После сортировки массив удовлетворял условию возрастания.

Для каждого теста выполнялось по несколько запусков, в том числе на больших массивах (до 10000 элементов). Все результаты подтвердили корректность реализованных алгоритмов.

Анализ допущенных ошибок

В процессе разработки были выявлены и исправлены следующие ошибки:

1. Неправильный подсчёт числа перемещений. Первоначально при каждом обмене двух элементов счётчик 'mov' увеличивался на 3 (по числу присваиваний). Однако в соответствии с заданием перемещением считается один обмен, а не отдельные присваивания. После исправления операция обмена стала увеличивать счётчик только на 1.
2. Лишние обмены в быстрой сортировке при разбиении. В реализации схемы Ломута при 'i != j' обмен не требовался, но в исходном коде обмен выполнялся всегда, что приводило к завышению числа перемещений на уже упорядоченных массивах. Добавлена проверка 'if (i != j)' перед обменом, а также аналогичная проверка при установке опорного элемента на место, чтобы избежать обмена элемента с самим собой.
3. Сброс глобальных счётчиков между запусками. Изначально забывали обнулять 'cmp' и 'mov' перед каждым вызовом сортировки, что приводило к накоплению статистики от предыдущих запусков. Ошибка устранена обнулением счётчиков непосредственно перед вызовом сортирующей функции.

Все перечисленные ошибки были исправлены, итоговая программа работает корректно во всех предусмотренных режимах.

Список литературы

- [1] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. Второе издание. — М.: «Вильямс», 2005.
- [2] Вирт Н. Алгоритмы и структуры данных. — М.: Мир, 1989.
- [3] Трифонов Н. П., Пильщиков В. Н. Задания практикума на ЭВМ (1 курс). Методическая разработка. — М.: ВМК МГУ, 2001.